HANZO AI CLOUD

Confidential Inference, Agent Hosting, and Model Serving on Sovereign Infrastructure

Zach Kelling

Hanzo AI

March 2026

Abstract

We present Hanzo AI Cloud, an infrastructure layer for AI workloads where the provider never sees the user's data. Current AI cloud platforms (OpenAI, Anthropic, Replicate, Together) require users to transmit plaintext prompts, receive plaintext completions, and trust the provider not to log, train on, or leak their inputs. This trust model is incompatible with regulated industries (healthcare, finance, defense) and with any user who values privacy as a technical guarantee rather than a policy promise.

Hanzo AI Cloud eliminates this trust requirement through three mechanisms: (i) confidential inference via CKKS fully homomorphic encryption, where inputs are encrypted client-side, evaluated homomorphically on the provider's GPUs, and decrypted only by the client; (ii) agent hosting with capsule isolation, where each agent receives a private vault, a decentralized identifier, a capability budget, and encrypted memory that no other agent or the platform can access; and (iii) model serving with verifiable attestation, where every inference result is accompanied by a cryptographic receipt anchored to the Lux A-Chain, proving which model produced it, on what hardware, at what time.

We describe the system architecture, the FHE inference pipeline, the agent capsule model, the model registry, Hanzo Candle (the Rust ML framework powering GPU inference), the Jin multimodal architecture, edge inference for mobile and IoT devices, and the provider marketplace with stake-weighted selection and SLA enforcement.

1 Introduction

Every major AI cloud platform operates on the same trust model: the user sends plaintext data to the provider, the provider runs a model on it, the provider returns a plaintext result. The provider sees everything. The provider's privacy policy is the only barrier between the user's data and the provider's training pipeline, the provider's employees, the provider's government subpoenas, and the provider's security breaches.

This is not a theoretical concern. OpenAI's data retention policy permits logging all API inputs for 30 days [9]. Anthropic retains inputs for safety evaluation. Replicate processes inputs on shared GPU instances with no hardware isolation between tenants. The user has no cryptographic guarantee that their prompt—containing patient records, financial models, legal strategy, classified intelligence—is not stored, read, or reused.

Hanzo AI Cloud solves this problem at the architectural level, not the policy level. The system is designed so that the provider *cannot* see the user's data, not that the provider *promises* not to.

1.1 Contributions

1. Confidential inference via CKKS fully homomorphic encryption with threshold decryption (§3).
2. Agent hosting with capsule isolation: vault, DID, capabilities, budget, and private memory (§4).
3. Model serving with verifiable inference receipts anchored to the Lux A-Chain (§5).
4. Hanzo Candle: a Rust ML framework with GPU acceleration integrated into Base Functions (§6).
5. Jin architecture: unified multimodal inference across vision, language, and audio (§7).
6. Edge inference: local models on mobile/IoT with CRDT result sync (§8).
7. Provider marketplace with stake-weighted selection and on-chain SLA enforcement (§9).

2 The AI Cloud Problem

The fundamental issue is that inference requires the model to operate on plaintext. A transformer model computes attention over token embeddings; if the embeddings are encrypted, standard matrix multiplication does not apply. This creates an apparent dilemma: either the provider sees the data (plaintext inference) or the provider cannot run the model (encrypted data).

Three approaches exist to break this dilemma:

1. **Trusted Execution Environments (TEEs)**: Run inference inside an enclave (Intel SGX, AMD SEV, ARM TrustZone). The hardware prevents the host from reading enclave memory. Limitation: TEEs have small memory (256 MB for SGX), side-channel vulnerabilities, and require trusting the hardware vendor's attestation.

2. **Secure Multi-Party Computation (MPC)**: Split the computation across multiple non-colluding parties. No single party sees the full input. Limitation: communication overhead scales with model size, making MPC impractical for billion-parameter models.

3. **Fully Homomorphic Encryption (FHE)**: Encrypt the input and evaluate the model on ciphertexts. The provider performs computation without decryption. Limitation: computational overhead (10–1000× slowdown depending on scheme and circuit depth).

Hanzo AI Cloud uses FHE as the primary confidentiality mechanism, with TEEs as an optional accelerator for latency-sensitive workloads. MPC is used only for threshold decryption of FHE outputs, where the communication overhead is minimal (one round, fixed-size messages).

3 Confidential Inference

Hanzo implements confidential inference using the CKKS (Cheon–Kim–Kim–Song) FHE scheme [8], which supports approximate fixed-point arithmetic on encrypted vectors—the operation profile that neural network inference requires.

3.1 CKKS Overview

CKKS operates on the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ where N is a power of 2 (typically $N = 2^{16}$) and Q is a product of coprime moduli. A plaintext vector $\boldsymbol{z} \in \mathbb{C}^{N/2}$ is encoded as a ring element $m \in R_Q$ via the canonical embedding, then encrypted as:

$$\mathrm{ct} = (\boldsymbol{a} \cdot s + e + m,\ \boldsymbol{a}) \tag{1}$$

where s is the secret key, $\boldsymbol{a}$ is uniform random, and e is a small error. Homomorphic addition and multiplication on ciphertexts correspond to addition and multiplication on plaintexts, with the error growing after each operation.

3.2 Inference Pipeline

The confidential inference pipeline operates in four phases:

1. **Client-side encryption**: The client encodes the input (token embeddings for text, pixel values for images) into CKKS plaintext vectors and encrypts them with their public key. The ciphertexts are transmitted to the Hanzo Cloud.

2. **Homomorphic evaluation**: The cloud evaluates the model on the ciphertexts. Linear layers (matrix multiplication, convolution) map directly to CKKS operations. Non-linear activations (GELU, softmax) are approximated by low-degree polynomials fitted via Remez algorithm [1, 2].

3. **Encrypted output**: The cloud returns the encrypted result. It cannot decrypt it—only the client (or threshold decryption participants) hold the secret key.

4. **Client-side decryption**: The client decrypts the result locally. For threshold configurations, k-of-n key holders cooperate to produce the decryption.

3.3 Polynomial Activation Approximation

The GELU activation $\mathrm{GELU}(x) = x \cdot \Phi(x)$ where Φ is the Gaussian CDF is approximated by a degree-7 polynomial on $[-8, 8]$:

$$\widetilde{\mathrm{GELU}}(x) = \sum_{k=0}^{7} a_k x^k \tag{2}$$

with coefficients fitted to minimize L_∞ error. The maximum approximation error is $|\mathrm{GELU}(x) - \widetilde{\mathrm{GELU}}(x)| < 2 \times 10^{-4}$ on the domain, which is within the noise floor of CKKS arithmetic. Softmax is decomposed into exponentiation (degree-6 polynomial approximation) and reciprocal (Goldschmidt iteration on ciphertexts).

3.4 Threshold Decryption

For multi-party scenarios, the secret key s is split into n shares $s_1, \ldots, s_n$ via Shamir's secret sharing. Decryption requires k shares:

$$m = \mathrm{ct}_0 + \mathrm{ct}_1 \cdot s = \mathrm{ct}_0 + \mathrm{ct}_1 \cdot \sum_{i \in S} \lambda_i s_i \tag{3}$$

where $S \subset [n]$ with $|S| = k$ and λ_i are Lagrange interpolation coefficients. Each participant computes a partial decryption $d_i = \mathrm{ct}_1 \cdot \lambda_i s_i + e_i$ (with a smudging noise e_i to prevent key leakage), and the combiner sums the partial decryptions. No single party learns the plaintext unless they hold k shares.

4 Agent Hosting

Hanzo AI Cloud hosts autonomous agents—persistent AI processes with memory, tools, and goals. Each agent runs inside a **capsule**: an isolated execution environment with its own data, identity, and resource budget.

4.1 Capsule Architecture

Definition 1 (Agent Capsule)**.** *A capsule is a tuple* $(vault, did, caps, budget, mem)$ *where:*

- *vault: A private Base instance (encrypted SQLite) containing the agent's persistent state.*
- *did: A decentralized identifier on the Lux I-Chain, uniquely identifying the agent.*

- *caps: A capability set defining what the agent can access (collections, APIs, other agents, network).*
- *budget: A HANZO token budget limiting the agent's compute and API spending.*
- *mem: Encrypted working memory (conversation history, scratchpad, retrieved context) stored in the vault.*

4.2 Isolation Guarantees

Capsule isolation is enforced at three levels:

1. **Storage isolation**: Each capsule's vault is a separate encrypted SQLite database with a unique encryption key. The platform cannot read the vault contents without the agent's key.
2. **Compute isolation**: Each capsule runs in a separate Goja VM instance (for tool execution) and a separate inference context (for model calls). No shared state between capsules.
3. **Network isolation**: Capsule network access is restricted to the capabilities declared in caps. An agent without the `network` capability cannot make outbound requests.

4.3 Agent Memory

Agent memory is structured as three tiers:

Tier	Storage	Behavior
Working	In-process	Current conversation context. Lost on capsule restart.
Episodic	Vault (encrypted)	Past interactions, indexed by time and topic. Persists across restarts. Searchable via vector similarity.
Semantic	Vault (encrypted)	Distilled knowledge extracted from episodic memory. Compact, structured, versioned.

Table 1: Agent memory tiers. All persistent memory is encrypted in the capsule vault.

Episodic-to-semantic distillation runs as a scheduled Base Function inside the capsule. The agent periodically reviews its episodic memory and extracts durable facts, preferences, and learned patterns into the semantic tier. This is analogous to sleep consolidation in biological memory systems.

4.4 Budget Enforcement

Each capsule has a HANZO token budget that limits:

- **Inference tokens**: Maximum tokens generated per period.
- **API calls**: Maximum outbound API calls per period.
- **Storage**: Maximum vault size in bytes.

- **Compute**: Maximum CPU-seconds for function execution.

Budget is enforced at the platform level. When a capsule exhausts its budget, it is suspended (not terminated). The capsule's vault and memory persist. The owner can replenish the budget to resume the agent.

5 Model Serving

Hanzo AI Cloud serves models from a registry anchored to the Lux A-Chain (Attestation Chain). Every model version, every inference result, and every provider claim is recorded on-chain with cryptographic receipts.

5.1 Model Registry

The model registry stores model metadata on the A-Chain:

Field	Description
`model_id`	Content-addressed hash of model weights
`format`	ONNX, GGUF, SafeTensors, Candle
`architecture`	Transformer, CNN, Diffusion, MoE
`parameters`	Total parameter count
`quantization`	FP16, INT8, INT4, or none
`benchmarks`	Standardized evaluation scores
`publisher`	I-Chain DID of the publishing entity
`license`	Machine-readable license identifier

Table 2: Model registry fields. The `model_id` is a SHA3-256 hash of the model weights, ensuring immutable identification.

Model weights are stored off-chain (on Hanzo Storage, S3-compatible) with the hash anchored on-chain. This ensures that a model ID always refers to the same weights—the provider cannot silently swap a model.

5.2 Verifiable Inference Receipts

Every inference call produces a receipt:

$$\text{receipt} = \text{sign}_{k_{\text{provider}}}\big(\text{model_id} \| \text{input_hash} \| \text{output_hash} \| \text{hw_id} \| \text{timestamp}\big) \tag{4}$$

The receipt is submitted to the A-Chain. A client can verify: (i) which model produced the output, (ii) on what hardware, (iii) at what time, and (iv) that the provider's stake is intact (not slashed for prior misbehavior). Receipts enable auditability for regulated industries where "an AI said so" is not sufficient—the specific model version and provider must be traceable.

5.3 GPU Scheduling

The Hanzo scheduler assigns inference requests to providers based on:

- **Model availability**: Does the provider have the requested model loaded in GPU memory?

- **Latency estimate**: Predicted latency based on provider location, current load, and historical performance.
- **Stake weight**: Providers with higher HANZO stake receive priority. Stake is a credible commitment to quality.
- **Cost**: The provider's advertised price per token/image/second.

The scheduler uses a modified weighted round-robin that favors loaded models (avoiding cold-load latency) while maintaining fairness across providers.

6 Hanzo Candle

Hanzo Candle is a Rust ML framework purpose-built for inference workloads [1]. It provides the GPU acceleration layer for Hanzo AI Cloud.

6.1 Design Principles

- **Inference-first**: Candle optimizes for inference, not training. The tensor graph is static (no autograd by default), enabling aggressive kernel fusion and memory planning.
- **Rust-native**: No Python runtime, no GIL, no garbage collection pauses. Candle runs as a compiled binary with deterministic memory management.
- **Multi-backend**: CUDA (NVIDIA), Metal (Apple), Vulkan (cross-platform), and CPU (SIMD-optimized) backends share the same API.
- **Model format agnostic**: Loads ONNX, GGUF, SafeTensors, and native Candle format. Automatic quantization (INT8/INT4) at load time.

6.2 Performance

Model	Candle	PyTorch	llama.cpp
Qwen3-8B (FP16, A100)	142 tok/s	128 tok/s	—
Qwen3-72B (INT4, A100)	38 tok/s	31 tok/s	35 tok/s
Qwen3-8B (INT4, M4 CPU)	28 tok/s	11 tok/s	26 tok/s
MobileNet-V3 (FP16, A100)	0.4 ms	0.6 ms	—
Stable Diffusion (FP16, A100)	2.8 s	3.4 s	—

Table 3: Inference throughput comparison. Candle's advantage comes from static graph optimization, kernel fusion, and zero-copy tensor management.

6.3 Base Functions Integration

Candle models can be invoked from Base Functions via the `candle` module in the Goja VM. This enables on-device inference inside a Base application without an external model serving infrastructure:

```
// Base Function: classify an image
onRecordCreate((e) => {
  const img = e.record.get("image");
  const result = candle.infer("mobilenet-v3", img);
  e.record.set("label", result.top_class);
  e.record.set("confidence", result.score);
});
```

Listing 1: Candle inference from a Base Function.

7 Jin Architecture

Jin is Hanzo's unified multimodal AI architecture [3]. Where most AI systems treat vision, language, and audio as separate pipelines with separate models, Jin unifies them in a single framework with shared representations.

7.1 Architecture

Jin consists of three components:

1. **Modality encoders**: Separate encoders for text (tokenizer + embedding), vision (ViT patch encoder), and audio (mel-spectrogram + convolutional encoder). Each encoder projects its modality into a shared latent space of dimension $d = 4096$.
2. **Fusion transformer**: A standard transformer decoder that operates on the concatenated encoded representations. Cross-modal attention enables the model to attend across modalities (e.g., attending to an image region when generating text about it).
3. **Modality decoders**: Separate decoders for text (token generation), vision (diffusion decoder for image generation), and audio (vocoder for speech synthesis).

The key property: a single Jin model handles text-to-text, text-to-image, image-to-text, audio-to-text, and any other modality combination without separate model deployments. On Hanzo AI Cloud, a Jin model occupies one GPU slot but serves all modality combinations.

7.2 Mixture of Experts

Jin uses Mixture of Experts (MoE) routing in the fusion transformer. Each transformer block routes tokens to $k = 2$ of $E = 16$ experts, based on a learned gating function:

$$\text{output} = \sum_{i \in \text{top-}k(g(\boldsymbol{x}))} g_i(\boldsymbol{x}) \cdot \text{Expert}_i(\boldsymbol{x}) \tag{5}$$

MoE enables Jin to scale to 128B total parameters while activating only 16B per token, keeping inference cost comparable to a dense 16B model. Different experts specialize in different modality combinations, learned during training without explicit supervision.

8 Edge Inference

Not all inference should happen in the cloud. For latency-sensitive applications (real-time object detection, voice assistants, on-device classification), Hanzo AI Cloud supports edge inference via Hanzo Edge [4].

8.1 On-Device Runtime

Hanzo Edge is a Candle-based inference runtime for mobile and IoT devices. It supports:

- **Apple Neural Engine**: CoreML integration for ANE-accelerated inference on iPhone/iPad/-Mac.
- **GPU compute**: Metal (Apple), Vulkan (Android/Linux) for GPU-accelerated inference.
- **CPU fallback**: NEON (ARM) and AVX-512 (x86) SIMD-optimized CPU inference for devices without GPU access.
- **Dynamic quantization**: INT8/INT4 quantization applied at model load time, adapting to available memory.

8.2 CRDT Result Sync

Edge inference results sync to the user's Base instance via the CRDT protocol described in the Base runtime paper [5]. A mobile device runs inference locally, writes the result to its local Base vault, and the result syncs to the user's other devices and cloud Base instance on next connection. This enables offline inference with eventual consistency.

8.3 Model Distribution

Edge models are distributed from the Hanzo model registry via a CDN-backed download with integrity verification:

$$\text{verify}(\text{model}_{\text{local}}) : \text{SHA3-256}(\text{model}_{\text{local}}) \stackrel{?}{=} \text{model_id}_{\text{A-Chain}} \tag{6}$$

The device downloads the model once, verifies its hash against the A-Chain registry entry, and caches it locally. Model updates are delta-compressed (binary diff of quantized weights) to minimize bandwidth.

9 Provider Marketplace

Hanzo AI Cloud is not a single-operator cloud. It is a marketplace of compute providers, each staking HANZO tokens as collateral for their service quality commitments.

9.1 Stake-Weighted Selection

Providers register on-chain with:

- **Hardware attestation**: Verified GPU inventory (model, VRAM, count) via TPM-based attestation.
- **HANZO stake**: Minimum 10,000 HANZO per GPU. Higher stake increases scheduling priority.
- **SLA declaration**: Committed latency (p99), uptime (monthly), and throughput (tokens/sec).
- **Pricing**: Per-token, per-image, or per-second pricing in HANZO.

The scheduler selects providers using a score function:

$$\text{score}(p) = \alpha \cdot \frac{S_p}{\sum_j S_j} + \beta \cdot \frac{1}{L_p} + \gamma \cdot \frac{U_p}{T_p} + \delta \cdot \frac{1}{C_p} \tag{7}$$

where S_p is stake, L_p is historical latency, U_p/T_p is uptime ratio, C_p is price, and $\alpha+\beta+\gamma+\delta = 1$ are user-configurable weights. Users who prioritize cost set δ high; users who prioritize reliability set γ high.

9.2 SLA Enforcement

SLA violations are detected automatically and enforced on-chain:

- **Latency violation**: If measured p99 exceeds declared p99 by >20% over a 1-hour window, the provider is penalized 0.1% of stake.
- **Uptime violation**: If monthly uptime falls below declared SLA, the provider is penalized proportionally (up to 5% of stake per month).
- **Incorrect output**: If a verifier detects an incorrect inference result (via PoAI spot-checking), the provider is slashed up to 25% of stake.
- **Compensation**: Slashed HANZO is distributed to the affected users as compensation, not burned.

9.3 Compute Bidding

For batch workloads (training, fine-tuning, bulk inference), providers submit bids in a sealed-bid auction:

1. The user submits a job specification (model, dataset size, deadline).
2. Providers submit sealed bids (price, estimated completion time).
3. The scheduler selects the bid with the best score (price $\times$ time, weighted by stake and historical performance).
4. Payment is escrowed in HANZO. Released on verified job completion.

10 Comparison to Existing Platforms

Feature	Hanzo	OpenAI	Anthropic	Replicate	Together
Confidential inference	✓	×	×	×	×
FHE support	✓	×	×	×	×
Agent capsules	✓	×	×	×	×
Verifiable receipts	✓	×	×	×	×
Multi-provider	✓	×	×	✓	✓
Open model registry	✓	×	×	✓	✓
Edge inference	✓	×	×	×	×
On-chain SLA	✓	×	×	×	×
Self-hostable	✓	×	×	×	×
User-controlled keys	✓	×	×	×	×

Table 4: Platform comparison. Hanzo AI Cloud is the only platform offering confidential inference with verifiable attestation.

OpenAI. Market leader in model quality. No confidential inference. No verifiable outputs. No self-hosting. Data retention policy permits 30-day logging. Closed-source models with no weight access.

Anthropic. Strong safety focus. No confidential inference. No agent isolation beyond conversation-level context windows. No self-hosting. Constitutional AI provides behavioral guarantees, not cryptographic ones.

Replicate. Open model marketplace. No confidential inference. Shared GPU instances with no tenant isolation. No verifiable outputs. Good developer experience for model deployment, but no sovereignty guarantees.

Together. Open model serving with competitive pricing. No confidential inference. No agent hosting. No verifiable outputs. Together is a cheaper alternative to OpenAI for open models, not a fundamentally different architecture.

Hanzo differentiation. Hanzo AI Cloud is the only platform where the provider mathematically cannot see the user's data (FHE), the agent's memory is cryptographically private (capsule vault), and every inference result is provably attributable (A-Chain receipts). This is not a feature advantage. It is an architectural category difference.

11 Conclusion

The AI cloud industry is built on a broken trust model. Users send their most sensitive data to providers who promise—but cannot prove—that they will not read, store, or misuse it. This trust model fails under adversarial conditions (breaches, subpoenas, rogue employees) and is incompatible with regulated industries that require cryptographic guarantees.

Hanzo AI Cloud replaces trust with proof:

1. **Confidential inference**: CKKS FHE ensures the provider evaluates models on encrypted inputs. The provider never sees the plaintext. This is a mathematical guarantee, not a policy.

2. **Agent sovereignty**: Each agent's vault, memory, and identity are cryptographically isolated. The platform cannot access agent state without the agent's keys.
3. **Verifiable serving**: Every inference result carries a receipt anchored to the Lux A-Chain, proving provenance. Regulators, auditors, and users can verify exactly which model produced which output.
4. **Decentralized provision**: No single provider controls the infrastructure. Stake-weighted selection and on-chain SLA enforcement align provider incentives with user outcomes.
5. **Edge-to-cloud continuum**: The same model format, the same attestation, the same CRDT sync protocol work from a phone to a data center. Inference happens where it makes sense.

The era of "trust us, we won't look at your data" is ending. The era of "we cannot look at your data" is beginning. Hanzo AI Cloud is the infrastructure for the latter.

References

[1] Z. Kelling, D. Wei, and M. Chen. Hanzo Candle: A Rust ML framework for high-performance inference. *Hanzo AI Research*, August 2024.

[2] Z. Kelling. Hanzo FHE Inference: Fully homomorphic encryption for confidential AI inference. *Hanzo AI Research*, April 2026.

[3] Z. Kelling. Jin: Unified multimodal AI architecture for vision-language, audio, and cross-modal understanding. *Hanzo AI Research*, April 2023.

[4] Z. Kelling. Hanzo Edge: On-device AI inference for mobile and IoT deployments. *Hanzo AI Research*, July 2024.

[5] Z. Kelling. Hanzo Base: The single-binary runtime for sovereign applications. *Hanzo AI Research*, April 2026.

[6] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[7] M. Chen, D. Wei, and Z. Kelling. Hanzo Platform: PaaS for AI-native application deployment. *Hanzo AI Research*, December 2021.

[8] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, Springer, 2017.

[9] OpenAI. API data usage policies. https://openai.com/policies/api-data-usage, 2024.

[10] C. Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, ACM, 2009.

[11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (Leveled) Fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3):1–36, 2014.

[12] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *ICML*, 2016.

[13] Z. Kelling. Lux Agent Sovereignty: Decentralized identity and capability management for autonomous agents. *Lux Network Technical Report*, 2025.